Open Source Software Audit Project

Name: Ruchicha Bharti

Registration Number: 24BAC10044

Course: Open Source Software

Topic: Python

Slot: B22

Date of Submission: 31/3/2026

Origin and Philosophy

The Problem This Software Was Created to Solve

The Programming Gap of the late 1980s

In the late 1980s, the world of software development was deeply divided into two camps that rarely met in the middle. On one side, there were high-level "professional" languages like C and Fortran. These were the industry standards, but they were incredibly difficult to master. A developer had to manage memory manually and write hundreds of lines of "boilerplate" code just to perform simple tasks like opening a file or printing a list. For engineers working on the Amoeba Distributed Operating System, this complexity was a massive bottleneck.

On the other side of the spectrum were "scripting" or "teaching" languages like ABC and Bourne Shell. While ABC was revolutionary for its readability—it was the first language to use indentation to group blocks of code—it was a closed system. It could not talk to the underlying hardware easily, it lacked a standard library, and it was hard to extend for real-world engineering tasks.

Guido van Rossum's Christmas Experiment

Guido van Rossum, a researcher at the Centrum Wiskunde & Informatica (CWI) in the Netherlands, found himself trapped in this gap. He was tired of the "mental friction" required to write utilities in C, but he loved the clean, human-centric philosophy of the ABC language he had helped develop.

In December 1989, looking for a hobby project to keep him busy during the Christmas break, Guido decided to write a new language. His goal was simple but ambitious: create a language that possessed the readability of ABC but the extensibility of C. He wanted a tool that would allow a programmer to build a prototype in an hour that would take a C developer a week.

Why Python Was Shared Freely

The decision to release Python as open-source rather than keeping it a proprietary internal tool of CWI was a pivotal ethical choice. Guido realized that for a programming language to be truly successful, it required a massive "ecosystem" of libraries. If Python were proprietary, it would have died with the Amoeba project. By choosing the Open Source model, he invited the global community to become co-authors of the language.

This allowed Python to "stand on the shoulders of giants. " Instead of one company building every feature, developers from around the world contributed modules for networking, math, and data science . Today, this choice is why a student at VIT can use the same world-class AI tools like TensorFlow that a senior engineer at NASA uses - all because Guido chose to build in the open.

The Philosophy of the "Fun" Tool

The very name Python was a statement against the stiff, academic culture of 1980s computing. Named after Monty Python's Flying Circus , it signaled that programming should be accessible, experimental , and even slightly irreverent. Guido wanted to lower the barrier to entry, ensuring that a biologist , an artist, or a student could use code to solve their problems without needing a PhD in computer science .

The License — What it Actually Says

To properly audit Python, I located the official LICENSE file within the Python source distribution. The Python Software Foundation (PSF) License is a "permissive" open-source license, which is a critical philosophical distinction compared to the "copyleft" models like the GPL.

The Four Freedoms and the PSF License

The concept of "Free Software" is anchored in four essential freedoms defined by the Free Software Foundation:

1. **Freedom 0:** The freedom to run the program as you wish, for any purpose.
2. **Freedom 1:** The freedom to study how the program works, and change it so it does your computing as you wish.
3. **Freedom 2:** The freedom to redistribute copies so you can help others.
4. **Freedom 3:** The freedom to distribute copies of your modified versions to others.

Based on my reading of the PSF License text, it does grant all four freedoms. It explicitly allows users to "reproduce, analyze, test, perform and/or display publicly, prepare derivative works, distribute, and otherwise use Python" alone or in combination with other software.

Commercial Use and Proprietary Modifications

A recurring question in open-source ethics is whether a company can take the software, modify it, and sell it without sharing their changes. Under the PSF License, the answer is yes.

Because the PSF license is "permissive" rather than "reciprocal," it does not include a "copyleft" clause. A company like Google or Netflix can take Python's source code, optimize it for their private internal servers, and they are not legally obligated to release those

specific optimizations back to the public. This flexibility is why Python is so popular in corporate environments; it allows businesses to innovate on top of a free foundation without risking their trade secrets.

GPL vs. MIT/PSF: Which to Choose?

The primary difference between the GPL (General Public License) and the MIT/PSF licenses is "reciprocity". The GPL is a copyleft license; it ensures that the software stays free forever by requiring that any derivative work also be released under the GPL. The MIT and PSF licenses, however, prioritize the freedom of the *user* to do whatever they want with the code, including making it proprietary.

If I were to build a new tool today, my choice would depend on my goal:

- I would choose GPL if I wanted to build a community-owned utility that I never want a corporation to "close off" or turn into a paid product.
- I would choose MIT/PSF if I wanted my tool to be adopted as widely as possible, even by commercial startups, because it removes the legal fear of having to open-source their entire business model later.

'Free as in Free Beer' vs. 'Free as in Freedom'

There is a vital distinction between these two concepts. 'Free as in Beer' (Gratis) refers to the price—I downloaded Python for zero cost. 'Free as in Freedom' (Libre) refers to the rights.

An example from Python is the math library. If Python were only 'Free Beer,' I could use the math functions for free, but I couldn't see how they are calculated. Because Python is 'Free as in Freedom,' I can open the source code of the math module on my Linux machine, verify the algorithms myself, and even modify how a square root is calculated if I have a specific scientific need. I am not just a consumer; I am a co-owner of the tool.

The Ethics of Open Source

Open source is more than a legal framework or a way to get free tools; it is a fundamental statement about how human knowledge should be treated in a digital age. This section reflects on the moral tensions and societal impacts of the FOSS (Free and Open Source Software) movement.

Should All Software Be Open Source?

The debate over whether all software should be open is a conflict between transparency and sustainability.

- **The Argument For**: Proponents of universal open source argue that software has become the "infrastructure of modern life." Just as we would not trust a bridge built with secret materials, we should not trust critical digital infrastructure (like voting

systems or medical devices) that cannot be audited. Open source democratizes education, allowing any student to learn from world-class code.

- **The Argument Against**: Opponents argue that proprietary software is a valid way to protect intellectual property and fund massive research and development (R&D) budgets. Without the profit incentive of closed-source software, many specialized tools for niche industries might never be built because there would be no financial way to sustain the full-time experts needed to maintain them.

Profit on Top of Community Labor

A major ethical dilemma in the FOSS world is the "Parasite vs. Symbiote" relationship between corporations and volunteers.

Companies like Google, Red Hat, and Meta build billion-dollar empires using Linux and Python—tools built largely by community volunteers. Is it ethical to build a profitable product entirely on free labor?

- I believe this is only ethical if a Symbiotic Relationship exists. For example, while Google uses Python for free, it also hires "Core Developers" to work on the language full-time and provides significant funding to the Python Software Foundation.
- However, when a company uses FOSS code to make billions but provides zero financial or technical support back to the project, they are effectively "mining" community labor. This can lead to maintainer burnout and the eventual collapse of critical tools.

Standing on the Shoulders of Giants

In software, 'standing on the shoulders of giants' means that we do not have to reinvent the wheel every time we write a script.

- **Collaborative Innovation**: Every Python library I use is a "shoulder" provided by a developer who came before me. Because they shared their work, I can focus on solving new, higher-level problems.
- **Enable vs. Undermine**: I argue that open source enables original innovation rather than undermining it. In the proprietary world, innovation is often slow because teams are busy re-writing basic functions that are hidden behind competitors' patents. In the FOSS world, the "basics" are shared, which forces developers to innovate on the truly new and difficult parts of technology.

Linux Footprint

1. Installation Method

- For my project audit, I chose to work within a WSL2 Ubuntu environment on my laptop. Since Python is a foundational piece of the open-source world, it was already present, but I wanted to ensure I was using the latest stable version managed by the community.
- I used the Advanced Package Tool (apt), which is the standard FOSS package manager for Debian-based systems. Instead of compiling from source—which is powerful but time-consuming—using apt allows the Ubuntu security team to push community-verified patches directly to my machine. I simply ran sudo apt update followed by sudo apt install python3 to finalize the setup.

```
ruchicha_bharti@LAPTOP-OI64CRF1:~$ sudo apt install python3
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
python3 is already the newest version (3.12.3-0ubuntu2.1).
python3 set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 209 not upgraded.
```

2. Key Directories

- After the installation, I used the which and where is commands to map out exactly where Python "lives" in the Linux file system hierarchy:
- **Binaries**: The actual executable file is located at /usr/bin/python3.
- **Standard Libraries**: I found that the core modules that give Python its power are stored in /usr/lib/python3.10/ (or your specific version).

```
ruchicha_bharti@LAPTOP-OI64CRF1:~$ /usr/bin/python3
Python 3.12.3 (main, Nov  6 2024, 18:32:19) [GCC 13.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
```

3. User and Group Security

- A critical part of this audit was understanding the permissions model. Even though I used sudo to install the software, the Python interpreter itself runs under my own user account.
- This matters immensely for security. By running as a standard user rather than "root," the software is restricted by the Principle of Least Privilege. If a script I write has a major error or security flaw, it only has permission to modify my personal files and cannot accidentally delete or corrupt essential system files in /etc or /boot.

```
ruchicha_bharti@LAPTOP-OI64CRF1:~$ ls -l /usr/bin/python3
lrwxrwxrwx 1 root root 10 Nov 12 12:15 /usr/bin/python3 -> python3.12
```

```
ruchicha_bharti@LAPTOP-OI64CRF1:~$ whoami
ruchicha_bharti
```

4. Service Management

- Unlike a background database or web server, Python is an interpreted language that I usually run on-demand. However, for this unit, I researched how Python scripts can be managed as background services using systemd.
- To check if a Python-based service is running, I would use: systemctl status my_script.service.
- To stop a running process safely: sudo systemctl stop my_script.service.

```
ruchicha_bharti@LAPTOP-OI64CRF1:~$ sudo systemctl list-units --type=service | grep -i "python"
ruchicha_bharti@LAPTOP-OI64CRF1:~$ sudo systemctl status
● LAPTOP-OI64CRF1
    State: running
    Units: 363 loaded (incl. loaded aliases)
     Jobs: 0 queued
   Failed: 0 units
    Since: Mon 2026-03-30 16:57:12 UTC; 53min ago
  systemd: 255.4-1ubuntu8.4
```

5. Update Model

- The beauty of the FOSS ecosystem is the community-driven security pipeline. When a vulnerability is found, the Python Software Foundation releases a fix, which is then picked up by the Ubuntu maintainers. I don't have to wait for a corporate "Patch Tuesday"; I simply run sudo apt upgrade to keep my environment secure.

```
ruchicha_bharti@LAPTOP-OI64CRF1:~$ sudo apt update
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
209 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

```
ruchicha_bharti@LAPTOP-OI64CRF1:~$ sudo apt upgrade --simulate
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following packages were automatically installed and are no longer required:
  libdrm-nouveau2 libdrm-radeon1 libgl1-amber-dri libglapi-mesa libllvm17t64 libwayland-server0 libxcb-dri2-0
```

The Foss Ecosystem

1. Dependencies: What Python Sits On

While Python is a high-level language, its reference implementation (CPython) depends on lower-level open-source tools to function.

- **C Libraries**: Python is written in C11, meaning it depends on the GNU C Library (glibc) on Linux for system calls and memory management.
- **Compilers**: To build Python from source or to compile C-extensions (like NumPy), you need the GNU Compiler Collection (GCC) or Clang.

2. Inspiration: What Python Enabled

Python's simple syntax inspired a "Cambrian explosion" of tools that have defined modern computing.

- **AI & Data Science**: Projects like TensorFlow, PyTorch, and scikit-learn have made Python the industry standard for Artificial Intelligence.
- **Automation**: Ansible, a leading open-source configuration management tool, is built entirely in Python to automate server deployments.

3. Web Context: Python and the LAMP Stack

The LAMP stack is the classic architecture for the web: Linux, Apache, MySQL, and PHP/Perl/Python.

- **The "P" in LAMP**: While PHP was the original "P," Python has become a popular replacement for building dynamic backends using frameworks like Django or Flask.
- **Integration**: Python connects the web server (Apache) to the database (MySQL/MariaDB) to serve dynamic content to users.

4. Community and Governance

Python is not owned by a company; it is governed by the Python Software Foundation (PSF), a non-profit.

- **Governance**: Since the creator (Guido van Rossum) stepped down as the "Benevolent Dictator," the language is now managed by an elected Steering Council.
- **Channels**: The community lives on discuss.python.org, various mailing lists (like python-dev), and IRC channels (e.g., #python on Libera.Chat).
- **Conferences**: PyCon is the primary global event where developers meet to share knowledge.

Open Source vs Proprietary

Dimension	Open Source (Python)	Proprietary Alternative (MATLAB)
Cost	Completely free for all users (student, commercial, academic).	Very high licensing costs; thousands of dollars for commercial use.
Security (who can audit the code?)	Anyone. The source code is publicly available for community scrutiny.	Only the company (MathWorks). Users must trust their internal security team.
Support and reliability	Community-driven. Support comes from forums (StackOverflow), PEPs, and IRC.	Professional corporate support with dedicated helpdesks and SLAs.
Freedom to modify	Total freedom. Users can modify the core interpreter under the PSF license.	No freedom to modify the underlying source code; strictly locked.
Community vs Corporate Control	Governed by the non-profit PSF and community consensus.	Controlled entirely by MathWorks for-profit interests.
Your Overall Verdict	Excellent for general-purpose development and massive-scale AI.	Superior for specific, high-end industrial control and simulation.

Conclusion and Recommendation

Given the comprehensive audit I have performed, I would highly recommend Python for a real-world deployment. Its permissive PSF license provides a low barrier to entry for

startups, and its massive ecosystem of community-built libraries—such as NumPy and TensorFlow—ensures that it can handle virtually any modern computing task without the "vendor lock-in" associated with proprietary tools. While MATLAB remains a powerful specialized tool, Python's adaptability and transparency make it the more sustainable choice for long-term growth.

I would certainly contribute to the Python project. The philosophy of "standing on the shoulders of giants" is only possible if developers contribute back to the pool of knowledge they use. Whether it is by reporting a bug on the PSF mailing lists or creating a new utility for the community, contributing to Python is a professional way to ensure that the tools of the future remain open, ethical, and accessible to everyone regardless of their financial status.

Shell Script Tasks

Script 1 — System Identity Report

Code:

```
#!/bin/bash
# Script 1: System Identity Report
# Author: ruchicha_bharti
# Course: Open Source Software (OSS NGMC)
# Purpose: Displays system metadata and licensing info as per Capstone requirements.

# --- Variables (Requirement: variables)
STUDENT_NAME="ruchicha bharti"
SOFTWARE_CHOICE="Python"

# --- System info gathering (Requirement: command substitution $())
# I used uname and hostnamectl to fetch the kernel and distro details
KERNEL_VER=$(uname -r)
USER_NAME=$(whoami)
UPTIME_VAL=$(uptime -p)
# Extracting the Operating System line from hostnamectl
DISTRO_NAME=$(hostnamectl | grep "Operating System" | cut -d ':' -f2)
CURRENT_TIME=$(date "+%Y-%m-%d %H:%M:%S")

# --- Display Output (Requirement: echo and basic formatting)
echo "======================================================"
echo "          OSS CAPSTONE: SYSTEM IDENTITY REPORT        "
echo "======================================================"
echo "Student Name    : $STUDENT_NAME"
echo "Project Subject : $SOFTWARE_CHOICE"
echo "------------------------------------------------------"
echo "Current User    : $USER_NAME"
echo "Linux Distro    : $DISTRO_NAME"
echo "Kernel Version  : $KERNEL_VER"
echo "System Uptime   : $UPTIME_VAL"
echo "Report Gen Time : $CURRENT_TIME"
echo "------------------------------------------------------"
echo "PHILSOPHY NOTE: This system is built on the values of"
echo "sharing and transparency. Python is distributed under"
echo "the PSF License, granting all four FOSS freedoms."
echo "======================================================"
```

Output:

```
ruchicha_bharti@LAPTOP-OI64CRF1:~$ nano script1.sh
ruchicha_bharti@LAPTOP-OI64CRF1:~$ chmod +x script1.sh
ruchicha_bharti@LAPTOP-OI64CRF1:~$ ./script1.sh
=========================================================
          OSS CAPSTONE: SYSTEM IDENTITY REPORT
=========================================================
Student Name     : ruchicha bharti
Project Subject  : Python
---------------------------------------------------------
Current User     : ruchicha_bharti
Linux Distro     :  Ubuntu 24.04.1 LTS
Kernel Version   : 6.6.87.2-microsoft-standard-WSL2
System Uptime    : up 5 minutes
Report Gen Time  : 2026-03-30 17:02:30
---------------------------------------------------------
PHILSOPHY NOTE: This system is built on the values of
sharing and transparency. Python is distributed under
the PSF License, granting all four FOSS freedoms.
=========================================================
```

Explaination:

The primary purpose of this script is to provide a standardized "Welcome Screen" for the Open Source Audit , identifying both the student and the technical environment.

Script 2 — FOSS Package Inspector

Code:

```bash
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: ruchicha_bharti
# Course: Open Source Software

PACKAGE="python3"

# Check if package is installed [cite: 135]
if dpkg -l $PACKAGE &>/dev/null; then
    echo "$PACKAGE is installed."
    # Extract Version and License using pipes [cite: 139]
    apt-cache show $PACKAGE | grep -E 'Version|License' | head -n 2
else
    echo "$PACKAGE is NOT installed."
    exit 1
fi

# Case statement for philosophy note [cite: 141]
case $PACKAGE in
    python3) echo "Python: A language shaped by community consensus." ;;
    apache2) echo "Apache: The web server that built the open internet." ;;
    mysql*)  echo "MySQL: Open source at the heart of millions of apps." ;;
    *)       echo "A verified Open Source tool." ;;
esac
```

Output:

```
ruchicha_bharti@LAPTOP-OI64CRF1:~$ nano script2.sh
ruchicha_bharti@LAPTOP-OI64CRF1:~$ chmod +x script2.sh
ruchicha_bharti@LAPTOP-OI64CRF1:~$ ./script2.sh
python3 is installed.
Version: 3.12.3-0ubuntu2.1
Version: 3.12.3-0ubuntu1
Python: A language shaped by community consensus.
```

Explaination:

The goal of this script is to programmatically verify that the audited software—**Python**—is correctly installed and configured on the Linux system. This script acts as an automated "health check" before moving forward with the more complex parts of the audit.

Script 3 — Disk and Permission Auditor

Code:

```bash
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: ruchicha_bharti | Course: OSS

# Array of directories to audit [cite: 152]
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/home/ruchicha_bharti/.config")

echo -e "DIRECTORY\tPERMISSIONS\tOWNER\tSIZE"

# Loop through the list [cite: 154]
for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        # Extract perms/owner using awk and size using cut [cite: 157, 148]
        PERMS=$(ls -ld "$DIR" | awk '{print $1}')
        OWNER=$(ls -ld "$DIR" | awk '{print $3}')
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)

        echo -e "$DIR\t$PERMS\t$OWNER\t$SIZE"
    else
        echo -e "$DIR\t[NOT FOUND]"
    fi
done
```

Output:

```
ruchicha_bharti@LAPTOP-OI64CRF1:~$ nano script3.sh
ruchicha_bharti@LAPTOP-OI64CRF1:~$ chmod +x script3.sh
ruchicha_bharti@LAPTOP-OI64CRF1:~$ ./script3.sh
DIRECTORY        PERMISSIONS      OWNER    SIZE
/etc     drwxr-xr-x       root     4.2M
/var/log         drwxrwxr-x       root     50M
/home    drwxr-xr-x       root     56K
/usr/bin         drwxr-xr-x       root     92M
/home/ruchicha_bharti/.config     [NOT FOUND]
```

Explaination:

The primary objective of this script is to automate a security and resource audit of the Linux environment where Python is hosted. By checking directory access and storage footprints, I can ensure that the system follows best practices for open-source deployment.

Script 4 — Log File Analyzer

Code:

```
#!/bin/bash
# Script 4: Log Scan | User: ruchicha_bharti
LOG="audit.log"
echo -e "ERROR: Fail\nINFO: Python OK\nERROR: Timeout" > $LOG
ERRS=0; PY=0

while read -r LINE; do
    [[ "$LINE" == *"ERROR"* ]] && ((ERRS++))
    [[ "$LINE" == *"Python"* ]] && ((PY++))
done < "$LOG"

echo "Log Summary for $LOG:"
echo "Errors: $ERRS | Python Events: $PY"
rm $LOG
```

Output:

```
ruchicha_bharti@LAPTOP-OI64CRF1:~$ nano script4.sh
ruchicha_bharti@LAPTOP-OI64CRF1:~$ chmod +x script4.sh
ruchicha_bharti@LAPTOP-OI64CRF1:~$ ./script4.sh
Log Summary for audit.log:
Errors: 2 | Python Events: 1
```

Explaination:

Its primary purpose is to simulate the real-world task of a system administrator who must monitor the health of an open-source application—in this case, **Python**—by scanning system logs for specific events or failures.

Script 5 — The Open Source Manifesto Generator

Code:

```
#!/bin/bash
# Script 5: The Open Source Manifesto Generator
# Author: ruchicha_bharti | Course: OSS (Unit 5)
# Purpose: Interactive tool to generate a personalized FOSS philosophy.

# --- Concept: Aliases (Requirement: Demonstrated via comment)
# In a real shell environment, I would set an alias for this script:
# alias manifesto='./script5.sh'
# This allows the user to run the audit quickly from any directory.

MANIFESTO_FILE="philosophy_statement.txt"

echo "======================================================"
echo "      OPEN SOURCE PHILOSOPHY GENERATOR (UNIT 5)       "
echo "======================================================"

# --- Concept: User Input (Requirement: read)
read -p "1. What is the most important value in Open Source? (e.g. Freedom): " VAL_1
read -p "2. Why does this value matter for a student at VIT?: " VAL_2
read -p "3. How will you contribute back to Python in the future?: " VAL_3

# --- Concept: String Concatenation & date (Requirement: built-in variables)
# Building a structured paragraph by joining the user's answers
STATEMENT="As of $(date +%F), my FOSS philosophy is built on $VAL_1. This is vital at VIT because $VAL_2.

# --- Concept: Writing to a file (Requirement: >)
# Using > to create a fresh file with a header and the concatenated string
echo "--- OFFICIAL OPEN SOURCE MANIFESTO ---" > $MANIFESTO_FILE
echo "Author: ruchicha_bharti" >> $MANIFESTO_FILE
echo "Generated on: $(date)" >> $MANIFESTO_FILE
echo "--------------------------------------" >> $MANIFESTO_FILE
echo "$STATEMENT" >> $MANIFESTO_FILE
echo "--------------------------------------" >> $MANIFESTO_FILE

echo "Success! Your personalized philosophy has been saved to $MANIFESTO_FILE"
echo "Displaying the generated statement below:"
cat $MANIFESTO_FILE
```

Output:

```
ruchicha_bharti@LAPTOP-OI64CRF1:~$ nano script5.sh
ruchicha_bharti@LAPTOP-OI64CRF1:~$ chmod +x script5.sh
ruchicha_bharti@LAPTOP-OI64CRF1:~$ ./script5.sh
=====================================================
      OPEN SOURCE PHILOSOPHY GENERATOR (UNIT 5)
=====================================================
1. What is the most important value in Open Source? (e.g. Freedom): Transparency
2. Why does this value matter for a student at VIT?: It allows us to learn from the best code
3. How will you contribute back to Python in the future?: By reporting bugs
Success! Your personalized philosophy has been saved to philosophy_statement.txt
Displaying the generated statement below:
--- OFFICIAL OPEN SOURCE MANIFESTO ---
Author: ruchicha_bharti
Generated on: Mon Mar 30 17:28:45 UTC 2026
--------------------------------------
As of 2026-03-30, my FOSS philosophy is built on Transparency. This is vital at VIT because It allows us to learn from the best code. To ensure the ecosyste
m stays healthy, I plan to By reporting bugs.
--------------------------------------
```

Explaination:

Its primary goal is to act as a "Manifesto Generator," capturing the user's specific values and formalizing them into a permanent record.

Conclusion

Honestly, after going through this whole audit, my main takeaway is that Python is just the gold standard for how open-source should actually work. Running these scripts really showed me that Python isn't just some random software I downloaded; it's a huge part of how my Linux system actually functions. I could literally track down where the binaries lived and check the permissions myself, which is something you just can't do with "black-box" proprietary apps. That kind of transparency is a massive deal for me because it means I'm actually the one in control of my own computer. If something breaks, I don't have to wait for a corporate patch—I have the freedom to go in and figure it out.

On a personal note, writing that Open Source Manifesto for Script 5 made me stop and think about why I'm even doing this. It's so easy to just "pip install" something and forget about it, but this project reminded me that there's a whole community behind the PSF License keeping these tools free for students like me at VIT . My biggest realization wasn't just about the code itself, but about the ethics of sharing and building on each other's work . I'm definitely sticking with Python for my future projects. I'd much rather work in an ecosystem that values collaboration and transparency over just making a profit. It just feels like a better way to be an engineer.